



PRA: Quantitative Project Risk Analysis in R

Paul Govan 
Independent Researcher

Abstract

The **PRA** package for R provides a unified, open-source implementation of the core analytical methods used in quantitative project risk analysis: the Second Moment Method (SMM), Monte Carlo simulation (MCS), earned value management (EVM), contingency and sensitivity analysis, Bayesian risk inference and updating, sigmoidal learning curves, and design structure matrices (DSM). Building on prior theoretical work that introduced causal networks and structural network measures for project risk (Govan 2014; Govan and Damnjanovic 2016, 2020), **PRA** operationalizes these frameworks in a minimal, well-tested R package available on CRAN. Version 0.4.0 further adds an experimental probabilistic network module for forward simulation and Bayesian conditioning along causal chains, and an AI agent layer that exposes all analytical functions as tools callable by large language models via slash commands, natural language queries, and a retrieval-augmented generation knowledge base. The package adheres to rOpenSci Statistical Software Review standards and is supported by a 640-test suite running on continuous integration across macOS, Windows, and Ubuntu.

Keywords: project risk analysis, Monte Carlo simulation, earned value management, Bayesian inference, design structure matrix, R.

1. Introduction

Quantitative risk analysis is a foundational discipline in project management, enabling organizations to assess schedule and cost uncertainty, identify high-impact risk drivers, and set evidence-based contingency reserves. Govan (2014) and Govan and Damnjanovic (2016) introduced the first implementation of causal networks for project risk analysis grounded in the resource-based view, demonstrating that organizational capabilities and shared resources—not just schedule uncertainty—are the structural pathways through which risks propagate across project tasks. Extending this work, Govan and Damnjanovic (2020) developed structural network measures that quantify the topology of project risk networks, enabling system-

Despite broad industry adoption of quantitative methods, open-source tooling for integrated project risk analysis in R remains fragmented. Proprietary platforms such as Oracle Primavera Risk Analysis and Palisade @RISK provide integrated workflows but impose licensing costs and limit reproducibility. Within R, no single package unifies Monte Carlo schedule risk, earned value management, Bayesian risk inference, sigmoidal learning curves, and dependency structure analysis under a common API. The **mc2d** package (Pouillot and Delignette-Muller 2010) provides general-purpose two-dimensional Monte Carlo simulation but has no project-specific constructs. The **sensitivity** package implements variance-based indices for general functions but does not integrate with schedule or cost models. EVM metrics are absent from CRAN entirely. Bayesian packages such as **rstan** and **brms** operate at a high level of generality and require substantial expertise to apply in project risk contexts.

The following table summarizes how **PRA** compares with the most closely related R packages and the dominant commercial platform.

Feature	PRA	mc2d	sensitivity	rstan/brms	@RISK	:	-----	:	--	:	----												
--	:	--	:	--	:	----	MCS schedule risk	Yes	Partial	—	—	Yes	EVM (11 metrics)										
Yes	—	—	—	Yes	Bayesian updating	Yes	—	—	Yes	—	Sigmoidal learning curves	Yes	—	—	—	—							
Probabilistic networks	Exp.	—	—	Yes	—	AI agent layer	Yes	—	—	—	—	Open-source (CRAN)	Yes	Yes	Yes	Yes	—	SRR-compliant	Yes	—	—	—	—

```
R> install.packages("PRA")
R> library("PRA")
```

The remainder of this article describes each analytical module in turn, with working code examples drawn from the same R session. Section~2 covers schedule and cost uncertainty methods (SMM, MCS, contingency, sensitivity). Section~3 describes earned value management. Section~4 covers Bayesian risk inference and updating. Section~5 describes sigmoidal learning curves. Section~6 covers design structure matrices. Section~7 introduces the experi-

mental probabilistic network module. Section~8 describes the AI agent framework. Section~9 presents an integrated case study. Section~10 concludes.

2. Schedule and Cost Uncertainty

Two complementary methods are provided for quantifying schedule and cost uncertainty: the Second Moment Method for rapid analytical estimates, and Monte Carlo simulation for full distributional analysis. Contingency extraction and sensitivity analysis complete the workflow.

2.1. Second Moment Method

The Second Moment Method (SMM) propagates task-level means and variances to project-level estimates using the Central Limit Theorem (Benjamin and Cornell 2000). For a project with n tasks:

$$E[X] = \sum_{i=1}^n E[X_i], \quad \text{Var}(X) = \sum_{i=1}^n \text{Var}(X_i) + 2 \sum_{i < j} \rho_{ij} \sigma_i \sigma_j$$

The `smm()` function takes vectors of task means and variances plus a correlation matrix and returns the total mean, variance, and standard deviation.

```
R> task_means <- c(10, 15, 20)   # weeks
R> task_vars  <- c(4,  9,  16)
R> cor_mat <- matrix(c(
+   1.0, 0.5, 0.3,
+   0.5, 1.0, 0.4,
+   0.3, 0.4, 1.0
+ ), nrow = 3, byrow = TRUE)
R>
R> result <- smm(task_means, task_vars, cor_mat)
R> cat("Total mean:", round(result$total_mean, 2), "weeks\n")
```

Total mean: 45 weeks

```
R> cat("Total SD:  ", round(result$total_std, 2), "weeks\n")
```

Total SD: 7.03 weeks

```
R> ci <- result$total_mean + c(-1, 1) * 1.96 * result$total_std
R> cat("95% CI:    [", round(ci[1], 1), ",", round(ci[2], 1), "] weeks\n")
```

95% CI: [31.2 , 58.8] weeks

SMM is useful when only first and second moments are available and a rapid estimate is needed. When task distributions are non-normal or accurate tail behavior is required, Monte Carlo simulation is preferred.

2.2. Monte Carlo Simulation

`mcs()` propagates uncertainty through a project network by drawing samples from user-specified task distributions—triangular, normal, or uniform—over a large number of iterations (Vose 2008). Task correlations are incorporated via Cholesky decomposition of the supplied correlation matrix.

```
R> num_simulations <- 10000
R> task_distributions <- list(
+   list(type = "normal",      mean = 10, sd = 2),
+   list(type = "triangular", a = 5, b = 10, c = 15),
+   list(type = "uniform",    min = 8, max = 12)
+ )
```

```
R> results <- mcs(num_simulations, task_distributions, cor_mat)
R> cat("Simulated mean:", round(results$total_mean, 2), "weeks\n")
```

Simulated mean: 38.6 weeks

```
R> cat("Simulated SD:  ", round(results$total_sd, 2), "weeks\n")
```

Simulated SD: 4.47 weeks

The `mcs()` function returns an S3 object of class "mcs" with a `print()` method that summarizes percentiles. The full simulated distribution is available in `results$total_distribution`. On a modern laptop, 10,000 simulations of a 10-task network complete in under one second; increasing `num_simulations` to 100,000 provides more precise tail estimates at roughly ten-fold additional runtime. The default of 10,000 balances accuracy and speed for most practical analyses.

```
R> hist(results$total_distribution,
+   breaks = 50, freq = FALSE,
+   main = "MCS: Total Project Duration",
+   xlab = "Duration (weeks)", col = "steelblue", border = "white"
+ )
R> lines(density(results$total_distribution), col = "tomato", lwd = 2)
R> abline(v = results$total_mean, lty = 2, lwd = 1.5)
```

2.3. Contingency Analysis

`contingency()` extracts a reserve buffer as the difference between a high-confidence percentile and the base (median) estimate—a standard approach in project risk practice (Project Management Institute 2021).

```
R> reserve <- contingency(results, phigh = 0.80, pbase = 0.50)
R> cat("Schedule contingency (P80 - P50):", round(reserve, 2), "weeks\n")
```

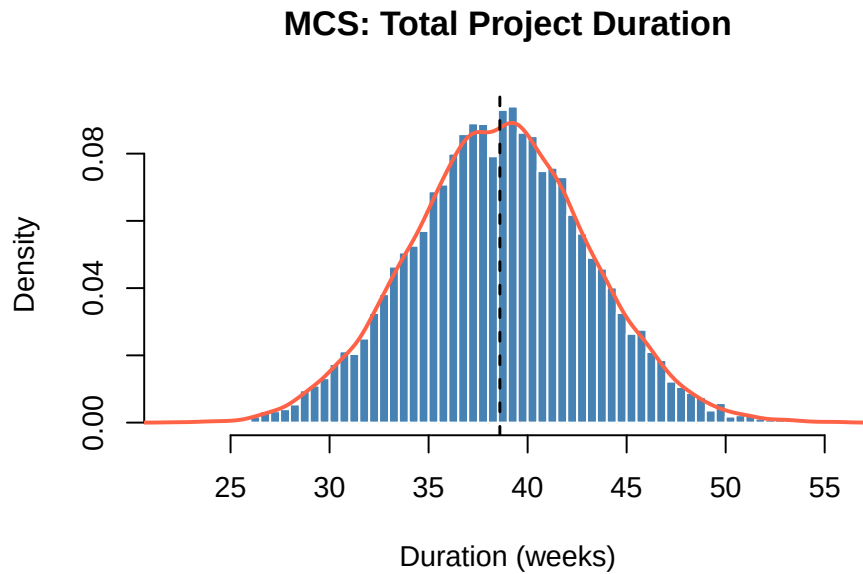


Figure 1: Distribution of simulated total project durations. The dashed vertical line marks the mean.

Schedule contingency (P80 - P50): 3.7 weeks

2.4. Sensitivity Analysis

`sensitivity()` decomposes total project variance by task contribution using rank correlation, producing the inputs for a Tornado chart.

```
R> sens <- sensitivity(task_distributions, cor_mat)
R> barplot(sort(sens),
+   horiz = TRUE, col = "steelblue",
+   main = "Sensitivity: Task Contributions",
+   xlab = "Rank Correlation with Total"
+ )
```

Tasks with larger bars are the dominant drivers of total risk; mitigation resources should be directed there first.

3. Earned Value Management

The EVM module implements the complete ANSI/EIA-748 suite of eleven performance metrics (Fleming and Koppelman 2010). The three core quantities—Planned Value (PV), Earned Value (EV), and Actual Cost (AC)—are computed first; all performance indices and forecasts follow from them.

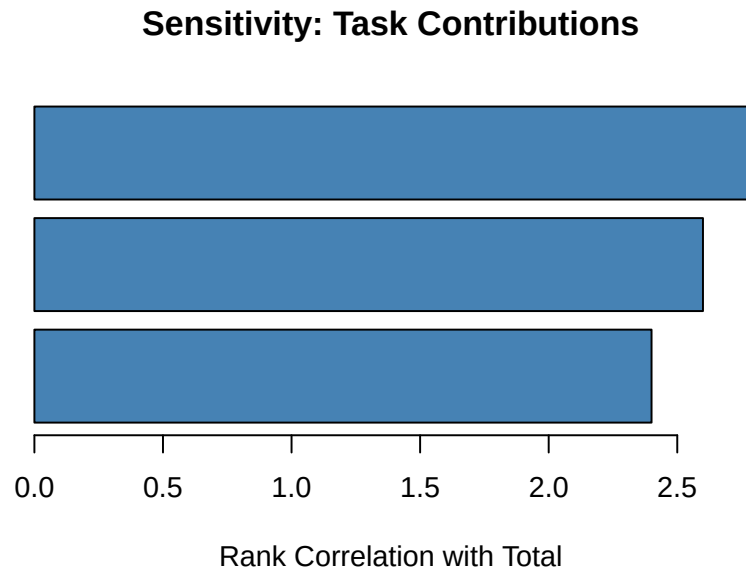


Figure 2: Tornado chart: each bar shows a task's rank correlation with total project duration.

```
R> bac          <- 500000                # budget at completion ($)
R> schedule     <- c(0.10, 0.25, 0.50, 0.75, 1.00) # cumulative value curve
R> time_period  <- 3

R> pv_val <- pv(bac, schedule, time_period)
R> ev_val <- ev(bac, actual_per_complete = 0.40)
R> ac_val <- ac(c(45000, 110000, 135000), time_period, cumulative = FALSE)
R>
R> cat("PV: $", format(pv_val, big.mark = ","), "\n")

PV: $ 250,000

R> cat("EV: $", format(ev_val, big.mark = ","), "\n")

EV: $ 2e+05

R> cat("AC: $", format(ac_val, big.mark = ","), "\n")

AC: $ 290,000

R> spi_val <- spi(ev_val, pv_val)
R> cpi_val <- cpi(ev_val, ac_val)
R> sv_val  <- sv(ev_val, pv_val)
R> cv_val  <- cv(ev_val, ac_val)
R>
R> cat("SPI:", round(spi_val, 3), " CPI:", round(cpi_val, 3), "\n")
```

SPI: 0.8 CPI: 0.69

```
R> cat("SV: $", format(round(sv_val), big.mark = ","),
+      "    CV: $", format(round(cv_val), big.mark = ","), "\n")
```

SV: \$ -50,000 CV: \$ -90,000

Both $SPI < 1$ and $CPI < 1$ indicate the project is behind schedule and over budget. The `eac()` function provides three completion forecasts under different efficiency assumptions:

```
R> cat("EAC (typical): $", format(round(eac(bac, method = "typical",
+      cpi = cpi_val)), big.mark = ","), "\n")
```

EAC (typical): \$ 725,000

```
R> cat("EAC (atypical): $", format(round(eac(bac, method = "atypical",
+      ac = ac_val, ev = ev_val)), big.mark = ","), "\n")
```

EAC (atypical): \$ 590,000

```
R> cat("EAC (combined): $", format(round(eac(bac, method = "combined",
+      cpi = cpi_val, ac = ac_val, ev = ev_val, spi = spi_val)),
+      big.mark = ","), "\n")
```

EAC (combined): \$ 833,750

The remaining EVM metrics are:

```
R> eac_typ <- eac(bac, method = "typical", cpi = cpi_val)
R> cat("ETC: $", format(round(etc(bac, ev_val, cpi = cpi_val)), big.mark = ","), "\n")
```

ETC: \$ 435,000

```
R> cat("TCPI:", round(tcpi(bac, ev_val, ac_val), 3), "\n")
```

TCPI: 1.429

```
R> cat("VAC: $", format(round(vac(bac, eac_typ)), big.mark = ","), "\n")
```

VAC: \$ -225,000

The To-Complete Performance Index (TCPI) reports the cost efficiency required on all remaining work to finish within the original budget; values substantially above 1.0 signal an unrealistic recovery target.

4. Bayesian Risk Methods

Building on the causal structure introduced in Govan (2014) and Govan and Damnjanovic (2016), the Bayesian module models risk event probabilities from root causes and supports formal updating as project observations become available (Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin 2013; Pearl 2009).

4.1. Prior Risk Probability

`risk_prob()` computes the marginal probability of a risk event R from multiple independent root causes using the total probability theorem:

$$P(R) = \sum_i P(R \mid C_i) P(C_i) + P(R \mid \bar{C}_i) P(\bar{C}_i)$$

```
R> cause_probs          <- c(0.30, 0.20)
R> risks_given_causes   <- c(0.80, 0.60)
R> risks_given_not_causes <- c(0.20, 0.40)
R>
R> prior <- risk_prob(cause_probs, risks_given_causes, risks_given_not_causes)
R> cat("Prior P(R):", round(prior, 3), "\n")
```

Prior P(R): 0.82

4.2. Bayesian Updating

`risk_post_prob()` updates the risk probability after observing cause status on site. Unobserved causes are passed as NA and are excluded from the update.

```
R> observed_causes <- c(1, NA) # Cause 1 present; Cause 2 not yet assessed
R> post <- risk_post_prob(cause_probs, risks_given_causes,
+                        risks_given_not_causes, observed_causes)
R> cat("Posterior P(R):", round(post, 3), "\n")
```

Posterior P(R): 0.632

```
R> cat("Change:", round(post - prior, 3), "\n")
```

Change: -0.188

Observing Cause~1 (which has a strong link to R) raises the risk probability substantially. The posterior informs contingency updates mid-project.

4.3. Cost Risk Distributions

`cost_pdf()` and `cost_post_pdf()` sample prior and posterior cost distributions by simulating which risks materialize and summing their cost impacts.

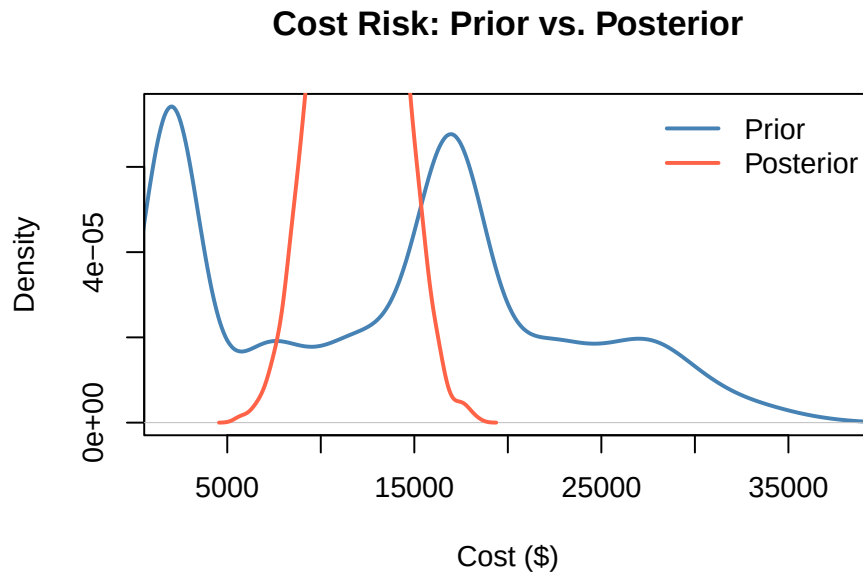


Figure 3: Prior and posterior cost distributions. Observing Cause 1 shifts the distribution rightward, increasing the expected cost impact.

```
R> risk_probs      <- c(0.3, 0.5, 0.2)
R> means_given_risks <- c(10000, 15000, 5000)
R> sds_given_risks  <- c(2000, 1000, 1000)
R> base_cost       <- 2000
R>
R> prior_costs <- cost_pdf(5000, risk_probs, means_given_risks,
+                          sds_given_risks, base_cost)
R> post_costs  <- cost_post_pdf(5000,
+   observed_risks = c(1, NA, 0), # Risk 1 present, Risk 2 unobserved, Risk 3 absent
+   means_given_risks, sds_given_risks, base_cost
+ )
R>
R> plot(density(prior_costs), col = "steelblue", lwd = 2,
+       main = "Cost Risk: Prior vs. Posterior",
+       xlab = "Cost ($)", xlim = range(c(prior_costs, post_costs)))
R> lines(density(post_costs), col = "tomato", lwd = 2)
R> legend("topright", legend = c("Prior", "Posterior"),
+       col = c("steelblue", "tomato"), lwd = 2, bty = "n")
```

4.4. Correlation Matrix Construction

`cor_matrix()` constructs a valid positive-definite correlation matrix from a vector of upper-triangle entries, for use as input to `mcs()`.

```
R> dists <- list(
+   normal = function(n) rnorm(n, mean = 10, sd = 2),
+   triang = function(n) mc2d::rpert(n, min = 5, mode = 10, max = 15),
+   uniform = function(n) runif(n, min = 8, max = 12)
+ )
R> C_emp <- cor_matrix(num_samples = 1000, num_vars = 3, dists = dists)
R> print(round(C_emp, 3))
```

```
      [,1] [,2] [,3]
[1,] 1.000 -0.046 0.015
[2,] -0.046 1.000 -0.005
[3,] 0.015 -0.005 1.000
```

5. Learning Curves

Sigmoidal (S-curve) models capture the characteristic pattern of project progress: slow start, rapid acceleration, and plateau approaching 100% completion. **PRA** provides three model types—Logistic, Pearl, and Gompertz—fitted via the Levenberg–Marquardt nonlinear least-squares algorithm in **minpack.lm** (Elzhov *et al.* 2016).

```
R> progress <- data.frame(
+   time = 1:9,
+   completion = c(5, 15, 40, 60, 70, 75, 80, 85, 90)
+ )

R> fit_log <- fit_sigmoidal(progress, "time", "completion", "logistic")
R> fit_gomp <- fit_sigmoidal(progress, "time", "completion", "gompertz")
R>
R> cat("Logistic RSE: ", round(summary(fit_log)$sigma, 2), "\n")
```

```
Logistic RSE: 4.15
```

```
R> cat("Gompertz RSE: ", round(summary(fit_gomp)$sigma, 2), "\n")
```

```
Gompertz RSE: 2.89
```

```
R> plot_sigmoidal(fit_log, progress, "time", "completion", "logistic",
+               conf_level = 0.95,
+               main = "Logistic Learning Curve",
+               xlab = "Week", ylab = "Completion (%)")
```

`predict_sigmoidal()` generates forecasts with confidence bounds at arbitrary future time points:

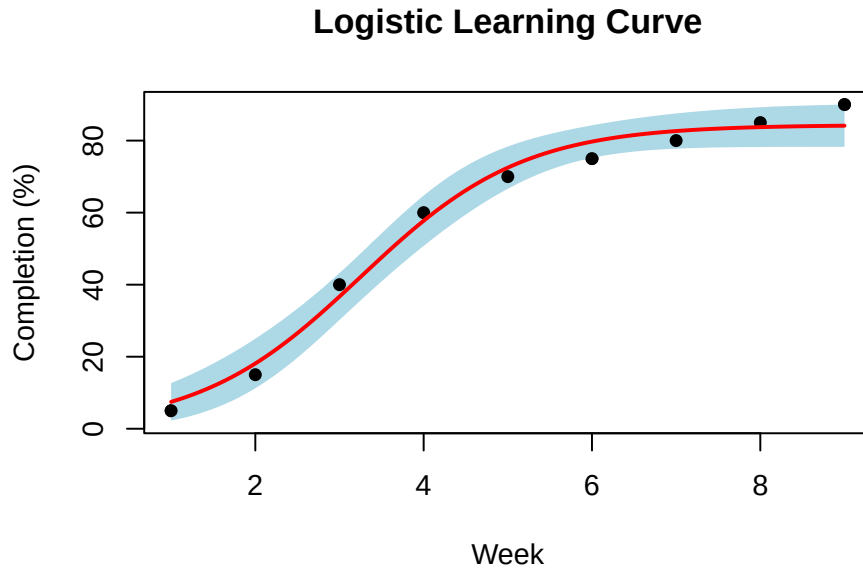


Figure 4: Logistic learning curve with 95% confidence band. The shaded region widens as extrapolation extends beyond the observed data range.

```
R> preds <- predict_sigmoidal(fit_log, seq(10, 12, by = 0.5),
+                             "logistic", conf_level = 0.95)
R> print(round(preds, 1))
```

	x	pred	lwr	upr
1	10.0	84.3	78.2	90.3
2	10.5	84.3	78.2	90.4
3	11.0	84.3	78.2	90.4
4	11.5	84.3	78.2	90.5
5	12.0	84.3	78.2	90.5

6. Dependency Structure Analysis

Building on Govan and Damnjanovic (2016, 2020) and the foundational DSM literature ([Steward 1981](#); [Browning 2001](#)), the DSM module quantifies structural coupling between project tasks through shared resources and risks.

6.1. Design Structure Matrices

The **Parent DSM** is derived from the Resource–Task matrix $\mathbf{S}$ (rows = resources, columns = tasks) as $\mathbf{P} = \mathbf{S}^\top \mathbf{S}$. Off-diagonal entry P_{jk} counts the number of resources shared between tasks j and k —the structural pathway through which disruptions propagate ([Govan and Damnjanovic 2016](#)).

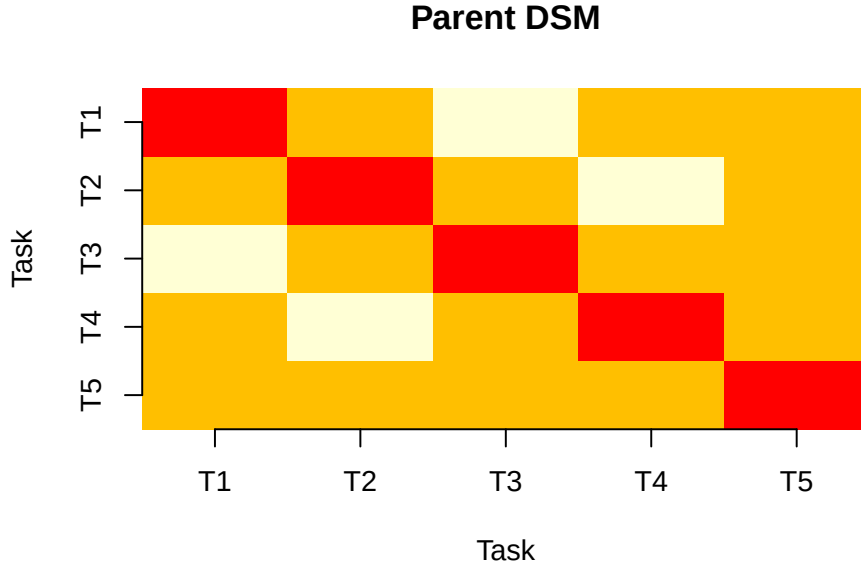


Figure 5: Parent DSM heatmap. Darker cells indicate more shared resources and stronger coupling between task pairs.

```
R> S <- matrix(c(
+   1, 0, 1, 0,
+   1, 1, 0, 0,
+   0, 1, 0, 1,
+   0, 0, 1, 1,
+   0, 1, 1, 0
+ ), nrow = 4, ncol = 5)
R> rownames(S) <- paste0("R", 1:4)
R> colnames(S) <- paste0("T", 1:5)

R> p <- parent_dsm(S)
R> plot(p)
```

The **Grandparent DSM** adds a risk layer via the Risk–Resource matrix **R**, tracing dependencies from risks through resources to tasks. Off-diagonal entry G_{jk} counts the number of risks shared between task j and task k through the resource chain—a direct operationalization of the structural network measures developed in Govan and Damnjanovic (2020).

```
R> R_mat <- matrix(c(
+   1, 0, 1,
+   1, 1, 0,
+   0, 1, 0,
+   0, 0, 1
+ ), nrow = 3, ncol = 4)
```

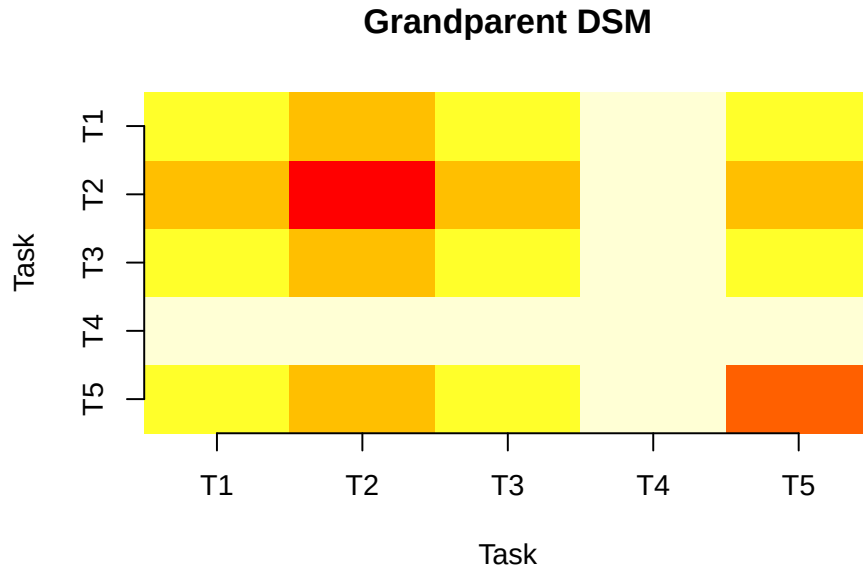


Figure 6: Grandparent DSM heatmap. Shows risk-based coupling: task pairs that share risk exposure through common resources tend to fail together.

```
R> rownames(R_mat) <- paste0("Risk", 1:3)
R> colnames(R_mat) <- paste0("R", 1:4)
```

```
R> g <- grandparent_dsm(S, R_mat)
R> plot(g)
```

Task pairs with high off-diagonal values in the Grandparent DSM represent concentrated risk exposure; these pairs are prime candidates for contingency buffering or resource decoupling.

7. Probabilistic Networks

Experimental. The probabilistic network module extends DSM structural analysis by attaching probability distributions to each node, enabling forward simulation of cost uncertainty and Bayesian conditioning on observed evidence. A probabilistic network is a directed acyclic graph in which nodes represent project variables (risks, resources, tasks, and the project total) and edges encode conditional dependencies (Pearl 2009; Govan 2014). Where a DSM counts shared resources between task pairs, a probabilistic network propagates distributions through the causal chain from risks to total project cost.

7.1. Building a Network

`prob_net()` constructs the network from a node data frame, a links data frame, and a named list of distributions. Distributions may be `normal`, `lognormal`, `uniform`, `discrete`, or

conditional (switching between two sub-distributions depending on a discrete parent node); aggregate nodes sum their parents.

```
R> nodes <- data.frame(
+   id = c("A","B","C","D","E","F","G","H","I"),
+   label = c("Risk-1","Risk-2","Resource-1","Resource-2","Resource-3",
+             "Task-1","Task-2","Task-3","Project"),
+   stringsAsFactors = FALSE
+ )
R> links <- data.frame(
+   source = c("A","B","C","D","E","F","G","H"),
+   target = c("C","D","F","G","H","I","I","I"),
+   stringsAsFactors = FALSE
+ )
R> distributions <- list(
+   A = list(type = "discrete", values = c(1, 0), probs = c(0.70, 0.30)),
+   B = list(type = "discrete", values = c(1, 0), probs = c(0.60, 0.40)),
+   C = list(type = "conditional", condition = "A",
+     true_dist = list(type = "normal", mean = 30000, sd = 8000),
+     false_dist = list(type = "normal", mean = 15000, sd = 3000)),
+   D = list(type = "conditional", condition = "B",
+     true_dist = list(type = "normal", mean = 80000, sd = 20000),
+     false_dist = list(type = "normal", mean = 50000, sd = 10000)),
+   E = list(type = "normal", mean = 20000, sd = 4000),
+   F = list(type = "aggregate", nodes = "C"),
+   G = list(type = "aggregate", nodes = "D"),
+   H = list(type = "aggregate", nodes = "E"),
+   I = list(type = "aggregate", nodes = c("F","G","H"))
+ )
R> net <- prob_net(nodes, links, distributions = distributions)
```

7.2. Forward Simulation

`prob_net_sim()` draws samples from every node in topological order and returns a data frame with one column per node.

```
R> sim_net <- prob_net_sim(net, num_samples = 10000)
R> cat("Mean project cost: $", format(round(mean(sim_net$I)), big.mark = ","), "\n")
```

Mean project cost: \$ 113,125

```
R> cat("P80 project cost: $", format(round(quantile(sim_net$I, 0.80)),
+   big.mark = ","), "\n")
```

P80 project cost: \$ 134,681

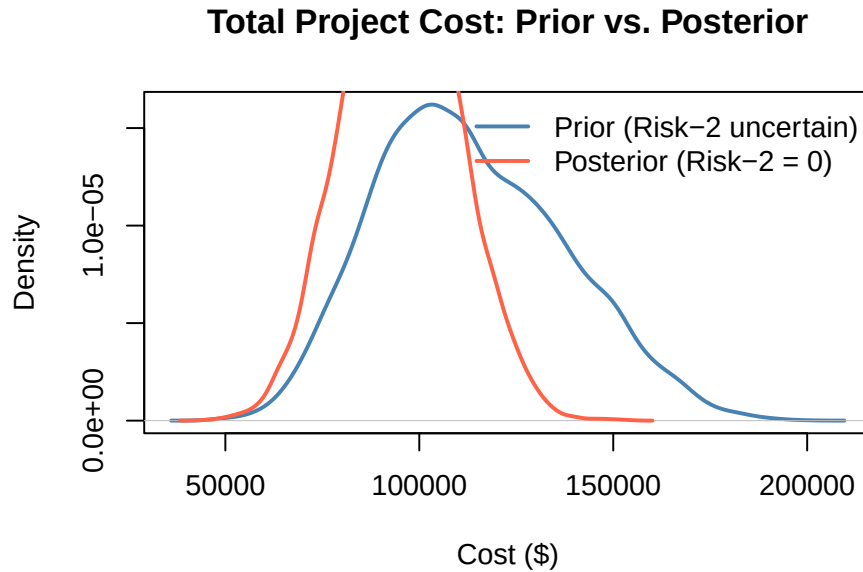


Figure 7: Total project cost before and after observing that Risk-2 (Technical Complexity) did not occur. Conditioning on Risk-2 = 0 eliminates the heavy upper tail and shifts the distribution leftward by roughly \$30,000 in expectation.

7.3. Learning: Conditioning on Evidence

`prob_net_learn()` clamps observed nodes to fixed values and re-simulates the downstream distribution—an observational update in the sense of Pearl (2009). The following call fixes Risk-2 (node B) to absent (value 0), ruling out Technical Complexity as a concern:

```
R> learn_net <- prob_net_learn(net, observations = list(B = 0),
+                               num_samples = 10000)
R>
R> plot(density(sim_net$I), col = "steelblue", lwd = 2,
+       main = "Total Project Cost: Prior vs. Posterior",
+       xlab = "Cost ($)")
R> lines(density(learn_net$I), col = "tomato", lwd = 2)
R> legend("topright",
+       legend = c("Prior (Risk-2 uncertain)", "Posterior (Risk-2 = 0)"),
+       col = c("steelblue", "tomato"), lwd = 2, bty = "n")
```

`prob_net_update()` performs structural surgery on the network by adding or removing edges and replacing node distributions, corresponding to the *do*-operator in Pearl’s causal calculus (Pearl 2009)—distinct from `prob_net_learn()`, which conditions without altering the graph. The probabilistic network API is marked experimental and may evolve in future releases.

8. AI Agent Framework

Version 0.4.0 of **PRA** includes an optional AI agent layer that wraps all analytical functions as tools callable by large language models. Three interaction modes are supported, summarized below.

Table: Agent routing modes in **PRA**.

Input type	Route	Example	:_____ :_____ :_____
—	/command syntax	Deterministic (no LLM)	/mcs n=10000 tasks=[...]
Numerical data	LLM tool call	"Simulate 3 tasks: Normal(10,2)..."	
Conceptual question	RAG knowledge base	"What is earned value?"	

1. **Slash commands** (/mcs, /evm, /risk, etc.) for deterministic, LLM-free execution via `PRA:::execute_command()`.
2. **Natural language** through `pra_chat()`, which connects to a local Ollama model (default: llama3.2) and uses **ellmer** (Wickham *et al.* 2025) tool-calling to route queries to the appropriate analytical function.
3. **Retrieval-augmented generation (RAG)** via **ragnar** (Posit PBC 2025): conceptual questions are answered from a bundled domain knowledge base with hybrid BM25 + vector search and source attribution.

An interactive Shiny application (`pra_app()`) integrates all three modes with a chat interface, data upload, and inline plots. `pra_mcp_server()` exposes all tools via the Model Context Protocol (Anthropic 2024), enabling integration with Claude, Claude Code, and other MCP-compatible clients. The server is registered in Claude Code with a single shell command:

```
R> claude mcp add -s project pra -- Rscript -e "PRA::pra_mcp_server()"
```

For Claude Desktop, add the following block to the MCP configuration file:

```
{
  "mcpServers": {
    "pra": { "command": "Rscript",
             "args": ["-e", "PRA::pra_mcp_server()"] }
  }
}
```

The bundled knowledge base covers all core methods. Analysts can extend it with project-specific documents using `build_knowledge_base()` and `add_documents()`:

```
R> store <- build_knowledge_base()
R> add_documents(store, "path/to/risk_register.md")
R> add_documents(store, "path/to/project_docs/")
```

The AI layer is strictly optional: all 36 exported analytical functions operate without any AI dependency. Tool definitions built with `ellmer::tool()` are reused identically for both the chat interface and the MCP server, eliminating duplication. A preliminary evaluation of 25

scenarios across single-tool, multi-tool, and open-ended query tiers yielded 72.0% overall accuracy (87.5% single-tool, 55.6% multi-tool, 75.0% open-ended), with the RAG layer providing the largest accuracy gain on open-ended conceptual queries. A full empirical evaluation is the subject of a forthcoming publication in *Risk Analysis*.

9. Illustrative Case Study

This section demonstrates an integrated, end-to-end project risk analysis for a three-task construction project, progressing through uncertainty quantification, performance monitoring, Bayesian risk updating, and dependency structure analysis in a single R session.

9.1. Step 1: Characterize Task Uncertainty

```
R> tasks <- list(
+   list(type = "triangular", a = 8, b = 12, c = 18), # earthworks
+   list(type = "normal", mean = 20, sd = 3), # structural frame
+   list(type = "triangular", a = 5, b = 8, c = 12) # MEP
+ )
R> C_proj <- matrix(c(1, 0.4, 0.2,
+                   0.4, 1, 0.3,
+                   0.2, 0.3, 1), nrow = 3, byrow = TRUE)
R>
R> sim <- mcs(10000, tasks, C_proj)
R> res <- contingency(sim, phigh = 0.80, pbase = 0.50)
R> cat("P50 schedule:", round(sim$percentiles["50%"], 1), "weeks\n")

P50 schedule: 51.3 weeks

R> cat("P80 schedule:", round(sim$percentiles["80%"], 1), "weeks\n")

P80 schedule: NA weeks

R> cat("Contingency (P80-P50):", round(res, 2), "weeks\n")

Contingency (P80-P50): 4.29 weeks
```

9.2. Step 2: Monitor Earned Value at Mid-Project

```
R> bac_cs <- 1200000
R> sched_cs <- c(0.15, 0.35, 0.60, 0.85, 1.00)
R> pv_cs <- pv(bac_cs, sched_cs, time_period = 3)
R> ev_cs <- ev(bac_cs, actual_per_complete = 0.50)
R> ac_cs <- ac(c(180000, 250000, 310000), time_period = 3, cumulative = FALSE)
R>
R> cat("SPI:", round(spi(ev_cs, pv_cs), 3),
+     " CPI:", round(cpi(ev_cs, ac_cs), 3), "\n")
```

SPI: 0.833 CPI: 0.811

```
R> cat("EAC (typical): $", format(round(eac(bac_cs,
+   method = "typical", cpi = cpi(ev_cs, ac_cs))), big.mark = ","), "\n")
```

EAC (typical): \$ 1,480,000

9.3. Step 3: Update Risk Probability After Site Inspection

```
R> prior_r <- risk_prob(c(0.25, 0.15), c(0.70, 0.50), c(0.15, 0.30))
R> post_r <- risk_post_prob(c(0.25, 0.15), c(0.70, 0.50),
+   c(0.15, 0.30), observed_causes = c(1, NA))
R> cat("Prior P(R):", round(prior_r, 3),
+   "-> Posterior P(R):", round(post_r, 3), "\n")
```

Prior P(R): 0.617 -> Posterior P(R): 0.609

9.4. Step 4: Identify Coupled Tasks via DSM

```
R> S_cs <- matrix(c(
+   1, 1, 0, 0,
+   0, 1, 1, 0,
+   1, 0, 0, 1,
+   0, 0, 1, 1,
+   0, 1, 0, 1
+ ), nrow = 4, ncol = 5)
R> R_cs <- matrix(c(
+   1, 0, 1,
+   0, 1, 0,
+   1, 1, 0,
+   0, 0, 1
+ ), nrow = 3, ncol = 4)
R> g_cs <- grandparent_dsm(S_cs, R_cs)
R> plot(g_cs)
```

The Grandparent DSM reveals which task pairs share risk exposure through common resources, informing targeted contingency allocation and resource decoupling decisions.

10. Summary

PRA is a CRAN-available R package that provides a unified, open-source implementation of quantitative project risk analysis methods. It operationalizes the causal network and structural network frameworks developed in Govan (2014) and Govan and Damjanovic (2016,

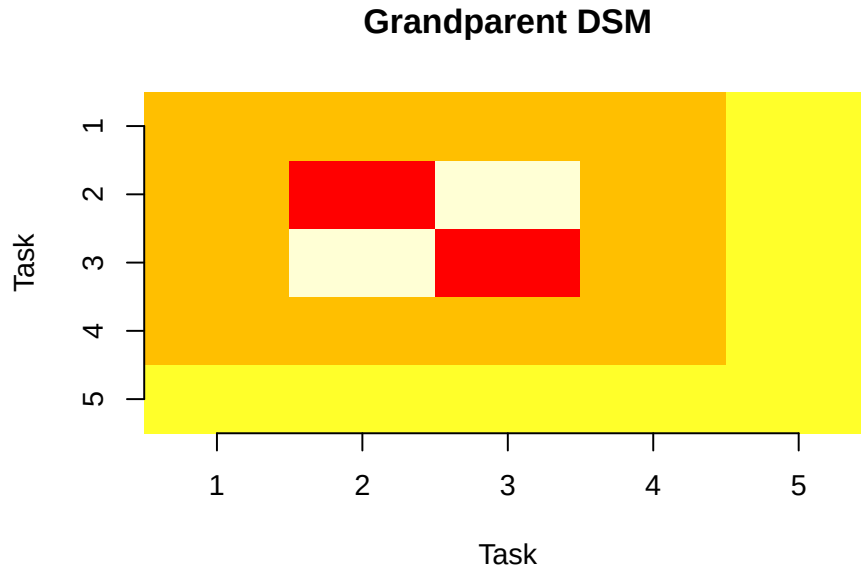


Figure 8: Grandparent DSM for the case-study project. Task pairs T3–T5 share the most risk exposure through common resources.

2020) through ten integrated analytical modules: SMM, MCS, contingency analysis, sensitivity analysis, EVM (11 metrics), Bayesian risk inference and updating, cost risk distributions, sigmoidal learning curves, correlation matrix construction, and design structure matrices. An experimental eleventh module provides probabilistic networks with forward simulation and Bayesian conditioning.

Version 0.4.0 adds an AI agent layer that exposes all functions as tools callable via slash commands, natural language, and RAG-powered queries through a local Shiny application and an MCP server. All 36 exported functions adhere to rOpenSci SRR standards, with 640+ tests verifying correctness, edge-case handling, and numerical stability across three operating systems. The package is licensed under MIT and the source is available at <https://github.com/paulgovan/PRA>; documentation and a companion online book are at <https://paulgovan.github.io/PRA/>.

10.1. Limitations and Future Work

Several limitations apply. First, **PRA** focuses on quantitative uncertainty propagation; it does not model schedule logic (critical path, activity dependencies, or resource leveling), which remain the domain of dedicated scheduling engines. Second, the probabilistic network module is experimental: the API may change and no convergence diagnostics are provided for the Monte Carlo estimators. Third, AI agent accuracy (72.0% on 25 scenarios) is preliminary; the evaluation set is small and the routing heuristics have not been stress-tested on large or ambiguous inputs. Fourth, the package currently accepts data as R vectors and lists; direct import from project management file formats (Microsoft Project, Primavera P6, CSV exports from Jira or Asana) is not supported and is planned for a future release.

A forthcoming companion article in *Risk Analysis* presents a full empirical evaluation of the AI agent framework, including a 25-scenario benchmark, RAG retrieval quality analysis, and a comparison to purely LLM-based assistants.

References

- Anthropic (2024). “Model Context Protocol.” URL <https://modelcontextprotocol.io>.
- Benjamin JR, Cornell CA (2000). *Probability, Statistics, and Decision for Civil Engineers*. Dover Publications, Mineola, NY.
- Browning TR (2001). “Applying the Design Structure Matrix to System Decomposition and Integration Problems: A Review and New Directions.” *IEEE Transactions on Engineering Management*, **48**(3), 292–306. doi:10.1109/17.946528.
- Elzhov TV, Mullen KM, Spiess AN, Bolker B (2016). *minpack.lm: R Interface to the Levenberg-Marquardt Nonlinear Least-Squares Algorithm Found in MINPACK*. URL <https://CRAN.R-project.org/package=minpack.lm>.
- Fleming QW, Koppelman JM (2010). *Earned Value Project Management*. 4 edition. Project Management Institute, Newtown Square, PA.
- Gelman A, Carlin JB, Stern HS, Dunson DB, Vehtari A, Rubin DB (2013). *Bayesian Data Analysis*. 3 edition. CRC Press, Boca Raton, FL.
- Govan P (2014). *A Resource-Based View on Project Risk Management*. Ph.D. thesis, The University of Texas at Austin.
- Govan P, Damnjanovic I (2016). “The resource-based view on project risk management.” *Journal of Construction Engineering and Management*, **142**(9), 04016034. doi:10.1061/(ASCE)CO.1943-7862.0001157.
- Pearl J (2009). *Causality: Models, Reasoning, and Inference*. 2nd edition. Cambridge University Press, Cambridge, UK.
- Posit PBC (2025). *ragnar: Retrieval-Augmented Generation for R*. URL <https://ragnar.tidyverse.org>.
- Pouillot R, Delignette-Muller ML (2010). “Evaluating Variability and Uncertainty in Microbial Quantitative Risk Assessment Using Two R Packages.” *International Journal of Food Microbiology*, **142**(3), 330–340. doi:10.1016/j.ijfoodmicro.2010.07.011.
- Project Management Institute (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th edition. Project Management Institute, Newtown Square, PA.
- rOpenSci Statistical Software Peer Review (2021). *Statistical Software Peer Review: Standards and Guidelines*. URL <https://stats-devguide.ropensci.org>.
- Steward DV (1981). *Systems Analysis and Management: Structure, Strategy and Design*. Petrocelli Books, New York, NY.

Vose D (2008). *Risk Analysis: A Quantitative Guide*. 3 edition. Wiley, Chichester.

Wickham H, et al. (2025). *ellmer: Chat with Large Language Models*. URL <https://ellmer.tidyverse.org>.

Affiliation: